Öğrenme fazları: 1 • Ön-öğretim | 2 • İçerik | 3 • Katılım | 4 • Değerlendirme | 5 • İzleme

1 • Ön-öğretim **Ön Bilgi:** Hf.6'nın Pydantic `BaseModel/KamelModel` sözleşmesi (`Etkinlik/Koltuk/Bilet`) ve `Depends(get_depo)` yer tutucusu bu hafta gerçek bir veritabanına bağlanır; Hf.7'nin `istemci/` klasörü değişmeden aynı API sözleşmesiyle konuşmaya devam eder; YG II'nin `CREATE TABLE/ SELECT/JOIN` ve ACID bilgisi bu haftanın ORM sorgularının temelidir.

» Ünite 1 — ORM Kavramı, Model↔Tablo, Oturum

ORM: Tekrarlanan Çeviriyi Otomatikleştirme

YG II'de her tablo için "satırı nesneye çevirme" kodunu elle, tekrar tekrar yazdınız — bir sütun eklediğinizde üç farklı yerde güncelleme gerekiyordu. ORM bu çeviriyi otomatikleştirir: bir Python sınıfının tanımı aynı anda hem tablo şemasını hem satır↔nesne çeviri mantığını üretir.

Bildirimsel Model: `Mapped/mapped_column`

`class EtkinlikDB(Base): __tablename__ = "etkinlikler"` ile başlayan bir model; her alan `Mapped[tip]` tip ipucu + `mapped_column(...)` çağrısıyla tanımlanır — Hf.6'nın Pydantic `BaseModel`'inin tip ipucundan doğrulama üretmesiyle kıyaslanabilir bir fikir; burada tip ipucundan bir sütun tanımı üretilir.

`engine` ve `Session`

`create_engine("sqlite:///bilet.db", echo=...)` veritabanına nasıl bağlanılacağını bilen, uygulama başına genelde tek bir nesnedir; `sessionmaker(bind=engine)` ile üretilen `SessionLocal`'dan her istek için yeni bir `Session` açılıp kapatılır — bir Session, bir dizi sorgu/değişikliğin `commit/rollback` ile sonlandığı çalışma birimidir.

`get_db()` ve ORM vs Pydantic Ayrımı

`yield` içeren `get_db()` bağımlılığı, hata olsa bile `finally`'de `db.close()` çalıştırır — Hf.6'nın `Depends(get_depo)` yer tutucusu böylece dolar. `EtkinlikDB` bir satırın Python temsilidir, JSON'a serileştirilmek için tasarlanmamıştır; `Etkinlik` (Hf.6'dan değişmeden) JSON sözleşmesidir — `Etkinlik.model_validate(etkinlik_db, from_attributes=True)` köprüsü birini ötekine çevirir.

> ⚠ Dikkat
>
> `create_all` yalnızca eksik tabloları oluşturur, mevcut bir tabloyu güncellemez; gerçek bir projede şema göçü Alembic adlı bir araçla sürüm sürüm kaydedilir — bu ders yalnızca adını anar.

`get_db` bağımlılık jeneratörü (`get_db_uretec` etiketi):

```python
def get_db() -> Generator[Session, None, None]:
    """Her istek için YENİ bir Session açar, istek bitince KAPATIR.

    `yield`'den ÖNCEKİ blok isteği İŞLEYEN koddan ÖNCE, `finally` bloğu
    isteği İŞLEYEN kod bittikten SONRA (hata OLSA bile) çalışır.
    """
    db = SessionLocal()
    try:
        yield db
    finally:
        db.close()
```

» Ünite 2 — İlişkiler (Etkinlik–Koltuk–Bilet)

Etkinlik 1—N Koltuk

`KoltukDB`'de `etkinlik_id: Mapped[int] = mapped_column(ForeignKey("etkinlikler.id"))` bir koltuğun tam olarak bir etkinliğe ait olduğunu bildiren yabancı anahtardır; `EtkinlikDB`'de `relationship(back_populates="etkinlik", cascade="all, delete-orphan")` bu bağı karşılıklı olarak tanımlar.

Koltuk 1—0..1 Bilet

İptal edilen biletler Hf.6'daki gibi silinmez, `durum='iptal'` olarak saklanır (denetim amaçlı) — bu yüzden `koltuk_id` üzerinde düz bir `UNIQUE` kısıtı yetersizdir, bir koltuk zaman içinde birden fazla bilet kaydına sahip olabilir.

Tembel vs İstekli Yükleme

varsayılan `lazy="select"` bir ilişkiye (`etkinlik.koltuklar`) ilk erişildiğinde ayrı bir SQL sorgusu tetikler; bir listedeki her kayıt için bu ayrı ayrı yapılırsa N+1 sorunu oluşur — `.options(selectinload(EtkinlikDB.koltuklar))` tüm ilişkileri tek bir ek sorguda önceden çeker.

Cascade Silme Kararı

`DELETE /etkinlikler/{id}` önce bağlı koltuklardan herhangi birinin aktif (`durum='olusturuldu'`) bileti olup olmadığını kontrol eder — varsa 409, yoksa `cascade="all, delete-orphan"` bağlı tüm koltukları (ve iptal biletlerini) otomatik siler.

Ortak Hata / Tuzak

Bir listedeki her kayıt için ilişkiye ayrı ayrı erişmek N+1 sorununa yol açar — listeleme uç noktalarında `selectinload` ile istekli yükleme kullanılmalıdır, varsayılan tembel yükleme bir liste için değil.

İlişki tanımları (`orm_koltuk_modeli` etiketi):

```python
class KoltukDB(Base):
    """Bir `koltuklar` satırının Python temsili — HER koltuk bir
       etkinliğe AİTTİR."""

    __tablename__ = "koltuklar"

    id: Mapped[int] = mapped_column(primary_key=True)
    etkinlik_id: Mapped[int] = mapped_column(ForeignKey("etkinlikler.id"
        ))
    sira: Mapped[int]
    koltuk_no: Mapped[int]
    durum: Mapped[str] = mapped_column(default="bos")

    etkinlik: Mapped["EtkinlikDB"] = relationship(back_populates="
        koltuklar")
    biletler: Mapped[list["BiletDB"]] = relationship(
        back_populates="koltuk", cascade="all, delete-orphan"
    )
```

Karmaşıklık (big-O)

N kayıtlık bir liste + tembel yüklemeyle her kayıt için ayrı bir ilişki sorgusu → $O(1+N)$ veritabanı gidiş-gelişi (1 ana sorgu + N ilişki sorgusu); `selectinload` ile tüm ilişkiler tek bir ek sorguda çekilir → $O(1)$ sabit sayıda ek sorgu, liste büyüklüğünden bağımsız.

Cascade/409 kararının kodu (`depo_etkinlik_sil_cascade` etiketi):

```python
def etkinlik_sil(self, etkinlik_id: int) -> str:
    """Etkinliği SİLER (cascade) ya da neden silinemediğini bildirir.

    Dönüş: `"silindi"` | `"bulunamadi"` | `"aktif_bilet_var"`. Bağlı
    koltuklardan HERHANGİ BİRİNİN aktif (`durum='olusturuldu'`) bir
    bileti varsa silme REDDEDİLİR; yoksa `relationship`'teki
    `cascade="all, delete-orphan"` sayesinde bağlı TÜM koltuklar (ve
    varsa iptal edilmiş biletleri) OTOMATİK olarak kademeli silinir.
    """
    etkinlik_db = self.db.get(EtkinlikDB, etkinlik_id)
    if etkinlik_db is None:
        return "bulunamadi"

    aktif_bilet_var = any(
        bilet.durum == "olusturuldu"
        for koltuk in etkinlik_db.koltuklar
        for bilet in koltuk.biletler
    )
    if aktif_bilet_var:
        return "aktif_bilet_var"

    self.db.delete(etkinlik_db)
    self.db.commit()
    return "silindi"
```

İstekli yüklemenin elle yazılmış `JOIN` karşılığı (`sql_koltuk_join` etiketi):

```sql
-- Bir etkinliği koltuklarıyla BİRLİKTE tek sorguda çekme (selectinload
    'ın
-- elle yazılmış JOIN karşılığı — Ünite 2'nin istekli yükleme örneği)
SELECT etkinlikler.id, etkinlikler.baslik, koltuklar.id AS koltuk_id,
    koltuklar.sira, koltuklar.koltuk_no, koltuklar.durum
FROM etkinlikler
JOIN koltuklar ON koltuklar.etkinlik_id = etkinlikler.id
WHERE etkinlikler.id = 1;
```

» Ünite 3 — Sorgular, Sayfalama, Ham SQL Karşılığı

`select()` API'si

`select(EtkinlikDB).where(EtkinlikDB.mekan == mekan)` YG II'nin `SELECT * FROM etkinlikler WHERE mekan = ?` cümlesinin Python nesnesi olarak ifadesidir; `db.execute(stmt).scalars().all()` ile çalıştırılıp sonuçlar alınır.

`where/order_by` Zincirleme

Filtre ve sıralama koşulları zincirlenebilir: `.where(...).order_by(EtkinlikDB.tarih)` — YG II'de ayrı ayrı yazdığınız cümle parçalarının Python tarafındaki karşılığıdır.

`limit/offset` Sayfalama

`GET /etkinlikler?sayfa=2&boyut=10` gibi isteğe bağlı (varsayılan `sayfa=1`, `boyut=20`) sorgu parametreleri mevcut liste uç noktalarına eklenir; dönüş tipi hâlâ bir dizidir — OpenAPI şeması böylece geriye uyumlu kalır.

`echo=True` ve `text()` Kaçış Kapısı

`echo=True` açıkken her `db.execute(...)` çağrısının arkasında çalıştırdığı gerçek SQL konsola basılır; ORM'in `select()` API'si rahat ifade edemediği karmaşık bir sorgu için `text("SELECT ...")` bir kaçış kapısıdır.

Hangi Sorgu API'si?

`db.get(EtkinlikDB, id)`: birincil anahtarla tek kayıt getirir (`None` dönebilir); `db.execute(select(...)).scalars().all()`: birden fazla kaydı liste olarak getirir; `.scalar_one_or_none()`: sorgunun tam olarak 0 ya da 1 satır döndürmesi bekleniyorsa kullanılır — birden fazla satır dönerse hata fırlatır.

Sayfalamanın ham SQL karşılığı (`sql_sayfalama` etiketi):

```sql
-- GET /etkinlikler?sayfa=2&boyut=10 karşılığı
SELECT etkinlikler.id, etkinlikler.baslik, etkinlikler.tarih,
     etkinlikler.mekan, etkinlikler.kapasite, etkinlikler.bilet_fiyati
FROM etkinlikler
ORDER BY etkinlikler.id
LIMIT 10 OFFSET 10;
```

» Ünite 4 — Transaction ve Eşzamanlı Rezervasyon

ACID ve `commit/rollback`

Bir `Session`'a yapılan `db.add(...)`/değişiklikler veritabanına hemen yazılmaz, `db.commit()` çağrılana kadar bir taslak hâlinde bekler; bir hata olursa `db.rollback()` tüm taslak değişiklikleri geri alır — bu haftanın odağı ACID'in atomiklik ve yalıtım özellikleridir.

Neden Şimdi Görünür Oldu

Hf.6'nın bellekteki `dict`'inde tek bir Python süreci istekleri pratikte sırayla işliyordu, çakışma neredeyse İMKÂNSIZDI; bu hafta veri bir dosyada yaşadığından ve gerçek eşzamanlı bağlantılar olabileceğinden, yarış durumu artık gerçek bir tehdittir.

Tek Transaction: Kontrol Et + Dolu Yap + Yaz

`POST /biletler`'in içinde (a) koltuğun `'bos'` olduğunu kontrol etme, (b) `durum`'u `'dolu'`'ya çevirme, (c) yeni `BiletDB` ekleme — bu üç adım tek bir `Session`/tek bir `commit` sınırında yapılır; adımlar arasında ayrı `commit` çağrılırsa araya başka bir istek girebilir.

İki Savunma Katmanı

varsayılan `BEGIN DEFERRED` bir yazma işlemine kadar kilit almaz; `BEGIN IMMEDIATE` transaction başlar başlamaz bir yazma kilidi alır — ikinci istek bekler ya da `"database is locked"` hatası alır. Bu uygulama kontrolü hatayla atlanırsa, kısmi `UNIQUE` indeks (`... WHERE durum = 'olusturuldu'`) bir `IntegrityError` fırlatır; rota bunu `409`'a çevirir — veritabanının kendisinin uyguladığı son güvence.

Kısmi indeksin ham SQL karşılığı (`sql_aktif_koltuk_unique_indeks` etiketi):

```sql
-- Bir koltuk için EN FAZLA bir AKTİF bilet kaydına izin veren kısmi
-- (koşullu) UNIQUE indeks — app/orm_modeller.py'deki Index(...)
     tanımının
-- ürettiği DDL
CREATE UNIQUE INDEX ux_biletler_aktif_koltuk
ON biletler (koltuk_id)
WHERE durum = 'olusturuldu';
```

Ortak Hata / Tuzak

`IntegrityError` yakalandıktan sonra `db.rollback()` çağırmayı unutmak: `Session` yarım kalan bir transaction'la "kirli" durumda kalır, sonraki sorgular da hata verir — yakala/dön kalıbı daima `except IntegrityError: db.rollback(); raise HTTPException(409, ...)` sırasını izlemelidir.

İpucu

`echo=True` loglarında `BEGIN`/`SELECT`/`UPDATE`/ `INSERT`/`COMMIT` sırasını takip edin — iki ayrı istek arasında bir `COMMIT` görmeden yeni bir `BEGIN` başlıyorsa, o istek öncekini bekliyor demektir (kilit çalışıyor).

`Depends` Zinciri Kapanıyor

`get_depo(db: Session = Depends(get_db)) -> Depo: return Depo(db)` — Hf.6'nın `depo: Depo = Depends(get_depo)` yazımı rota fonksiyonlarında tek karakter değişmeden kalır; değişen yalnızca `get_depo`'nun içindeki `dict` yerine bir veritabanı `Session` kullanılmasıdır — Hf.6'nın "Hf.8'de bu yer tutucu değişecek" notunun gerçekleştiği yer.

Tek transaction'ın ham SQL karşılığı (`sql_transaction_rezervasyon` etiketi):

```sql
-- POST /biletler işleyicisinin TEK transaction'da yaptığı üç adımın
-- (kontrol et + dolu yap + yaz) elle SQL karşılığı
BEGIN IMMEDIATE;

SELECT id, durum FROM koltuklar WHERE id = 1;
-- Uygulama kodu burada durum = 'bos' mu diye kontrol eder.

UPDATE koltuklar SET durum = 'dolu' WHERE id = 1;

INSERT INTO biletler (koltuk_id, kullanici_adi, satin_alma_zamani,
     durum)
VALUES (1, 'Ayşe Yılmaz', '2026-11-14T10:00:00', 'olusturuldu');

COMMIT;
```

» Komutlar

`pip install -r requirements.txt`	Bağımlılıkları (SQLAlchemy dahil) yükler
`uvicorn app.main:app --reload`	Sunucuyu 8000 portunda başlatır, `bilet.db` dosyasını oluşturur
`python -m app.tohum`	Örnek etkinlik/koltuk verisini yükler
`python -m pytest -q`	Testleri (bellek içi SQLite ile) çalıştırır
`python -m pytest tests/test_transaction.py -v`	Yalnız transaction testlerini ayrıntılı çalıştırır
`python -m pytest -k cascade`	Yalnız cascade adı geçen testleri çalıştırır
`npm run dev`	Vite geliştirme sunucusunu (istemci, değişmeden) başlatır

5 • İzleme **Nerede Kullanılır:** İlişkili nesneler bir liste için mi gerekiyor → `selectinload` kullan, tembel yüklemeye güvenme · Bir etkinlik silinecek → önce aktif bilet kontrolü, varsa `409` · Birden fazla veritabanı değişikliği tek bir iş kuralına aitse → tek `Session`/tek `commit` içinde yap · Uygulama kontrolü hatayla atlanırsa → kısmi `UNIQUE` indeks son güvence · ORM'in ürettiği SQL'i görmek istiyorsanız → `echo=True` · ORM'in ifade edemediği karmaşık bir sorgu → `text()` · Liste uç noktasına sayfa parametresi eklemek → dönüş tipini değiştirme, yalnızca isteğe bağlı `sayfa`/`boyut` ekle · Şema değişiyor → `create_all` yetmez, gerçek projede Alembic gerekir